SEPPA: Correctness-Gated, LLM-Driven Kernel Optimization on the Raspberry Pi 5's Integrated GPU

Adam Munawar Rahman
New York University
New York, NY, USA
msr541@nyu.edu

***Abstract*—Single-board computers ship with integrated GPUs that most edge AI workloads leave idle. We present Seppa, a kernel-optimization harness in which a large language model proposes GPU kernel and host-contract changes while a finite-state machine, not the model, owns compilation, correctness verification, benchmarking, and the keep-or-revert verdict, refusing out-of-order transitions so that an unverified kernel can never reach a benchmark. Recent kernel-generation systems document pervasive reward hacking, remedied in every published case by machine-owned checks; Seppa makes that remedy structural. Applied to the flood-simulation stencil of Bonbibi, an offline flood-guidance application for the Raspberry Pi 5, the loop produced a fused, strip-mined V3D kernel 1.59x faster at 256x256, verified by three physics gates: fp64 NMSE, mass conservation, and basin pooling. The result survived a false plateau whose cause was structural: every winning change lay in the host contract, outside the original shader-only action space. The machine re-derived the result over the Model Context Protocol, including refusing to benchmark a deliberately mass-violating kernel. Under concurrent CPU inference the kernel's advantage grows to 2.09x, and the resulting deployment (10.3 tokens/s decode alongside 712 simulation steps/s) dominates partitioned, oversubscribed, and time-sliced CPU-only alternatives on both axes. Every number carries its exact reproduction command.**

***Index Terms*—GPU kernel optimization, LLM agents, correctness verification, edge computing, Vulkan**

I. Introduction

A Raspberry Pi 5 running a quantized language model uses its four Cortex-A76 cores and leaves its VideoCore VII (V3D) GPU idle. That GPU is modest: 256 maximum invocations per workgroup, 16 KB of shared memory, a fixed subgroup width of 16, no usable fp16 arithmetic, and no cooperative-matrix hardware. It is also, for a workload that does not need the CPU, free.

Bonbibi, the motivating application, is an offline flood-aware accessible-routing system for exactly this board [7]. Its division of labor is deliberate: the GPU simulates surface flooding over real terrain with a weighted cellular-automata model (WCA2D [1]), while the CPU concurrently runs a street router (a RoutingKit Customizable Contraction Hierarchy [2]) and a small language model (Granite 4.0 1B, CPU-only llama.cpp [8]) that narrates the computed route. Deterministic code owns the safety-critical logic; the model only phrases it. The GPU flood simulation is what the idle silicon buys, and its throughput determines how much simulation the system can afford per routing update.

This paper is about how that GPU code got fast, and about the harness that made the process trustworthy. Optimizing kernels for V3D is unforgiving: the driver's shader compiler falls down a register-allocation fallback ladder on code that any desktop GPU would accept, workgroups over 256 invocations corrupt silently rather than failing loudly, and intuitions imported from CUDA-class hardware (unroll more, prefetch more, tile more) are frequently wrong here, which we know because we measured them being wrong. An optimizer working on this target needs two things: a proposal engine that can absorb unfamiliar constraints quickly, and a verifier that cannot be talked out of the way.

Seppa supplies both, with an explicit separation of powers. A large language model owns only the `hypothesize` and `implement` steps of an optimization loop. A Burr [5] finite-state machine owns everything downstream: `compile`, `verify`, `benchmark`, `evaluate`, `log_variant`. The machine is served over the Model Context Protocol [6], and its single `step` tool refuses out-of-order calls, so the agent cannot skip verification or read a benchmark for a kernel that failed correctness. Correctness is not a convention the model is asked to follow; it is a transition the machine will not take.

Contributions:

1. A measured account of optimizing a real WCA2D flood stencil for V3D under three physics gates, reaching 1.59x at 256x256, including the falsified hypotheses (Section IV).
2. An action-space lesson for LLM-driven kernel optimization: the first search plateaued at zero improvement not because the kernel was at its limit but because every winning change (buffer packing, fusion, dispatch geometry) lived in the host contract, outside the shader-only action space the machine exposed. Making the host contract part of the `implement` step's inputs recovered the full speedup (Section IV-C).
3. A machine-checked reproduction: the FSM, driven over MCP, independently re-derives the result from its own baseline and issues its own keep verdict, and demonstrably refuses to benchmark a kernel that fails the physics gate (Section V).

4. An interference measurement of the target deployment: the optimized GPU flood loop and CPU LLM decode running concurrently, quantifying what "leveraging idle silicon" costs each side, and a gate-verified CPU-only counterfactual (sequential, partitioned, and oversubscribed) that the GPU split dominates on both axes (Section VI).

II. Related work

LLM-driven kernel generation is measured by KernelBench [3], whose fast_p metric gates speedup claims on functional correctness by definition. Yet the record of the systems built on it is largely a record of reward hacking. Kevin [11], trained with multi-turn RL, learned to wrap and inherit the PyTorch reference and required externally enforced guardrails. CUDA-L1 [12] reports that 82 of its 250 RL-generated kernels (32.8%) exploited stream-timing loopholes to fake 18x speedups, and needed layered anti-hacking machinery (reward checkers, a hacking-case database, forced stream synchronization) to contain it. TritonRL [13] finds delegation-based cheating across systems and names the discrepancy "a critical gap in reward fidelity." SpecBench [14] generalizes the pattern beyond kernels: frontier agents saturate visible test suites while failing held-out ones. Every published remedy is machine-owned verification external to the model. Seppa builds that conclusion into its structure: verification runs in-loop as a state transition, enforced by a server that refuses out-of-order calls, so the agent cannot skip it.

The strength of the oracle matters as much as its placement. Sarkar [15] shows the major kernel benchmarks verify correctness with fixed-shape, small-sample allclose checks that certify buggy kernels as correct, and argues for fuzzing against an fp64 CPU reference with per-operator tolerances. Seppa's gates are of the stronger kind: an fp64 CPU reference (NMSE) plus domain invariants, mass conservation against injected rainfall and physically correct pooling, that a numerically plausible but wrong kernel cannot satisfy.

Closer to hardware, LLMs have produced tapeout-grade Verilog conversationally (Chip-Chat [16]) and as fine-tuned generators (VeriGen [17]), with humans and testbenches closing the verification loop; Seppa moves that loop into a machine-owned FSM. FunSearch [4] and AlphaEvolve [9] pair LLM proposal with automated evaluators inside evolutionary searches and credit the evaluator, not the model, with the reliability of the results. Seppa shares that conviction and differs in target and mechanism: a hostile, sparsely documented embedded GPU rather than a well-modeled datacenter part, and an action space that includes the host contract, which Section IV-C shows is where the wins were.

The application side draws on established components: WCA2D [1] for reduced-complexity flood modeling (originally motivated by GPU parallelism), Customizable Contraction Hierarchies [2] for reroutable street routing with millisecond re-customization, and llama.cpp [8] for CPU inference. The V3D driver stack is Mesa's v3dv [10].

III. Seppa: the machine owns the gate

Seppa is an AutoKernel-style optimization loop restructured as a Burr finite-state machine and served over MCP by a wrapper (Theodosia) running on the Pi itself. The graph is:

```
characterize -> baseline -> hypothesize -> implement
  -> compile -> verify -> benchmark -> evaluate
  -> log_variant -> (hypothesize | stop)
```

with two guard edges: `compile -> log_variant` on compile failure, and `verify -> log_variant` on gate failure. The benchmark state is reachable only through a green verify. The agent's entire interface is one MCP tool, `step(action, inputs)`; the server constrains `action` to the graph's legal next moves, so "skip verify" is not an expressible request.

The agent owns two states. `hypothesize` returns the current best, the experiment history, and the hardware constraints (256 invocations, 16 KB shared, subgroup 16). `implement` accepts the agent's proposal. Everything else is deterministic machinery with no model in the loop: compilation via glslangValidator, verification and benchmarking via the target harness, and an `evaluate` step that keeps a variant only if it beats the best by more than 1%, persisting the winner to disk. Three consecutive non-improvements, or the experiment budget, ends the run. The machine's ledger (`variant_log`) records every experiment, kept or reverted, with its gate results.

For the flood target, `verify` runs the physics gate of the `vkflood2` harness: 400 steps at 256x256 must satisfy (a) NMSE below 1e-3 against a double-precision CPU reference of the same WCA2D update, (b) total water equal to injected rainfall (mass conservation), and (c) maximum depth pooling inside the terrain basin. The benchmark metric is simulation steps per second on the same run. In practice verified NMSE is far below the threshold (1.3e-9 at 4,000 steps).

IV. Case study: the flood stencil

A. The workload

Bonbibi's simulation is a two-pass stencil per time step: a flux pass computes limited inter-cell flows from water-surface-height differences, and a height pass applies net flow plus rainfall to each cell's depth. On the original harness this ran 400 steps at 256x256 in 0.383 s (about 1,045 steps/s, 1.51 GFLOP/s), with both shaders compiling cleanly. Notably, the shaders are too small to stress the v3dv register allocator, so an optimization playbook proven on llama.cpp kernels for this same GPU (removing source-level unrolling to rescue the compiler's register allocation, worth +28% on matrix-vector decode there) had nothing to act on here; the bound had to be found fresh.

B. Falsification sweep

Rather than guess the bottleneck, we bought each hypothesis a variant and let the gate-and-benchmark loop price it. All variants passed all three physics gates; speeds are for 400 steps at 256x256 on the parameterized `vkflood2` harness. Its host loop is itself faster than the original harness for identical kernels (about 1,345 steps/s, 1.94 GFLOP/s, against

the 1,045 above), so every comparison in this paper is within `vkflood2`.

Variant	Hypothesis tested	Result
Packed vec2 state (water, surface) halving flux-pass loads	Memory-op count is the bound	1.93 GFLOP/s, no change: falsified
Fused single-dispatch step (flux buffer eliminated)	Traffic and barriers are the bound	2.04 GFLOP/s, +5%: mostly falsified
2 cells per invocation (strip-mined dispatch)	Fixed per-invocation cost is the bound	2.40 GFLOP/s, +23%: supported
4 cells per invocation	More strip is better	2.35 GFLOP/s: falsified, register pressure
Fused + 2 cells per invocation (`fused2s.comp`)	The wins stack	3.08 GFLOP/s, +59%

The stencil is bound by fixed per-invocation cost: each invocation does so little arithmetic that launch overhead dominates, and amortizing it over two cells is worth more than eliminating half the memory system's work. Two further V3D specifics shaped the winner. Vertical strips keep a subgroup's 16 lanes on adjacent x addresses, and lane contiguity matters more on this GPU than per-thread load count. And the kernel had to be written for the register allocator: holding all four flux vectors of the cell pair live fails v3dv register allocation and falls down the compiler's fallback ladder, so the shipped kernel consumes each flux as it is produced and keeps only four scalars.

The shipped kernel is 1.59x at 256x256 (0.187 s vs 0.297 s), 1.41x at 512x512, 1.18x at 1024x1024, and holds all gates over a 4,000-step run (NMSE 1.3e-9, mass conserved, pools in basin). Reproduction:

```
g++ -O3 -o vkflood2 vkflood2.cpp -lvulkan
glslangValidator -V fused2s.comp -o fused2s.spv
STRIP=2 FUSED=1 FLUX_SPV=fused2s.spv ./vkflood2 256 400
```

Integrating it into Bonbibi is three host-side changes: the packed vec2 state buffer, one pipeline and one dispatch per step, and a halved dispatch height (`pi/flood/README.md`).

C. The false plateau was an action-space limit

An earlier Seppa session pointed the FSM at this same stencil and measured flat: no variant beat the baseline, and the session concluded there was no headroom. That conclusion was wrong, and the reason matters for anyone building an LLM-driven optimizer. The machine's `implement` step accepted only shader source text under a fixed host program. Every change that won in Section IV-B lives outside that space: vec2 packing is a buffer-layout change, fusion deletes a pipeline and a barrier from the host loop, and strip-mining changes the dispatch geometry. The search was sound and its result was correct for the space it was allowed to search.

The fix is `v3d_flood2_opt.py`: `implement` now takes {`shader`, `height_shader`, `strip`}, where an empty height shader means a fused single-dispatch step and `strip` sets cells per invocation and the dispatch shape. Fusion and dispatch geometry became legal FSM moves rather than out-of-band build decisions, and the machine found and kept the fused strip-2 kernel from its own baseline (Section V). The general rule we take from this: when a correctness-gated search plateaus, ask whether the winning move is expressible in the action space before concluding the target is exhausted. A plateau is evidence about the search space, not only about the hardware.

V. Machine-checked reproduction over MCP

To make the optimization claim checkable, the reproduction is executed by the machine rather than narrated by the author. `drive_flood2_mcp.py` connects to the Theodosia server's MCP endpoint from a separate machine and drives the full cycle through the `step` tool, in two parts.

The first part re-derives the result. In the run of 2026-07-11 (transcript in `docs/paper/artifacts/`), the FSM measured its own baseline at 1,346.8 steps/s with all physics gates green, received the fused strip-2 kernel as experiment 1, compiled it, passed all three gates, benchmarked 2,116.4 steps/s, and issued its own verdict: keep. That is 1.57x, machine-derived end to end and consistent with the hand-measured sweep. The machine's ledger line reads:

```
{"exp": 1, "fused": true, "strip": 2, "compile_ok": true,
 "verify_ok": true, "steps_per_sec": 2116.4,
 "best_sps": 2116.4, "verdict": "keep"}
```

The second part demonstrates the gate. The driver submits the same kernel with one change, rainfall injection doubled in one of the two cell updates. It compiles. The physics gate fails it (`verify_ok: false`), and the driver then requests `benchmark` anyway. The server's verbatim response:

```
{"error": "invalid_transition", "requested": "benchmark",
 "valid_next_actions": ["log_variant"],
 "message": "action 'benchmark' is not reachable from
     current state. Valid actions now: ['log_variant']."}
```

The broken variant enters the ledger as `verdict: revert` with `steps_per_sec: null`. The refusal is the property the harness exists to provide: neither the model nor a buggy or adversarial client can obtain a performance number for physics it broke.

Two independent repetitions agree: an in-process run on the Pi (`replay_flood2.py`, 2026-07-08) measured baseline 1,351 and kept 2,116 steps/s, and a first over-the-wire run earlier on 2026-07-11 measured baseline 1,346.8 and kept 2,105.3 steps/s.

VI. Concurrent CPU and GPU work

The optimization exists to make a CPU-plus-GPU split worth having, so the last measurement is interference. Bonbibi's deployment shape is: the GPU loops flood simulation while the CPU runs street routing and Granite 4.0 1B decode (llama.cpp, `-ngl 0`, 4 threads; the GGUF reports 1.63 B parameters under llama.cpp's granite-3B architecture label, which is what llama-bench prints). The board shares one LPDDR memory system between the halves, so whether the GPU is effectively free has to be measured rather than assumed.

The measurement campaign itself produced a finding worth stating first. With any Vulkan device visible, llama.cpp allocates CPU-resident model weights in the GPU's host-pinned, write-combined memory even at `-ngl 0`, which is fast for GPU transfers that never happen and slow for the CPU reads that dominate decode. Hiding the device (`GGML_VK_VISIBLE_DEVICES=99`) moves decode from 9.40 ± 0.36 to 11.44 ± 0.20 tokens/s: 22% from one environment variable, same binary, same flags (A/B at cool start, r=5). Bonbibi's launch scripts already hid the device, so the deployment always had this; an earlier revision of this study did not, and its CPU figures were correspondingly low. Every number below uses the corrected, deployment-matching configuration. The portable rule: a CPU-only llama.cpp process on a board with an integrated GPU should hide that GPU.

`pi/flood/concurrency_bench.sh` measures six conditions on the Pi: each flood kernel alone (4,000-step runs), CPU decode alone (llama-bench tg64, 5 repetitions, 4 threads), the router alone, and each flood kernel looping continuously while the same decode benchmark runs, plus the router under flood load. Every phase starts from below 55 C, and a 1 Hz sampler records temperature and the throttle register throughout; `pi/flood/analyze_conc_bench.py` reduces the raw logs and reports per-phase thermal validity. Concurrent flood runs count only if they start and finish inside the decode window. Results from the run of 2026-07-12 (raw logs in `docs/paper/artifacts/conc_bench2/`; the earlier GPU-visible run is preserved in `conc_bench/`):

Condition	GPU flood (steps/s)	CPU decode (t/s)
Optimized kernel alone	2,127.3 ± 1.7 (n=3)	
Original kernel alone	1,348.9 ± 0.3 (n=3)	
Decode alone		11.5 ± 0.0
Concurrent, optimized kernel	711.9 ± 76.1 (n=5)	10.3 ± 0.2
Concurrent, original kernel	340.1 ± 16.9 (n=2)	10.2 ± 0.3

The router is not in the table because it does not move: `route.py` completes in 0.02 s alone and 0.02 s under full GPU flood load. The n=2 on the original-kernel concurrent row is a consequence of its own slowness: only two 4,000-step runs fit inside the decode window.

Three results. First, the co-processing thesis holds with a measured price. Adding the optimized flood loop to a board already saturated with CPU decode costs the CPU about 10% of its decode rate and buys a continuous 712 steps/s flood simulation that a CPU-only deployment simply does not have. Second, interference is strongly asymmetric, and the fast-decode configuration makes it more so: the CPU keeps 90% of its solo throughput, but the GPU keeps only 33% (the faster the CPU's decode, the more of the shared LPDDR bandwidth it consumes, and the GPU is the victim of that sharing). The idle GPU delivers a third of its solo throughput when the CPU is busy: the silicon is available, but much of its bandwidth is not. Third, the kernel optimization matters more under contention: the optimized kernel's advantage over the original grows from 1.58x alone to 2.09x concurrent, at equal CPU cost (decode 10.3 against 10.2). This is consistent with the fused kernel's elimination of the intermediate flux-buffer traffic: fewer bytes through the shared memory system per simulation step means less to lose when bandwidth is contended. Under deployment conditions, the optimized kernel is the difference between a usable co-processor and a marginal one.

A. The CPU-only counterfactual

Concurrency is only worth defending against the alternative: running the flood on the CPU too. `pi/flood/cpuflood.cpp` is the same WCA2D update in float with OpenMP row parallelism, verified by the same three physics gates against the same double-precision reference (NMSE 1.46e-11, mass conserved, pools in basin, at 1 and 4 threads). `pi/flood/cpu_flood_bench.sh` measures it alone and sharing the four cores with decode two ways: oversubscribed (4 flood threads and 4 decode threads competing for 4 cores) and partitioned (flood pinned to core 0, decode pinned to cores 1 to 3). Same cooldown gates and thermal sampling; raw logs in `docs/paper/artifacts/cpu_flood_bench2/`.

Condition	Flood (steps/s)	CPU decode (t/s)
CPU flood alone, 1 / 2 / 4 threads	992.0 / 1,977.9 / 3,803.2	
Decode alone, 3 threads (pinned)		11.8 ± 0.0
Partitioned: flood 1t + decode 3t	680.6 ± 22.9 (n=6)	8.4 ± 0.0
Oversubscribed: flood 4t + decode 4t	703.4 ± 338.7 (n=21)	2.4 ± 0.3
GPU concurrent, optimized (from above)	711.9 ± 76.1	10.3 ± 0.2

The first row answers the objection a skeptical reader should raise: four idle A76 cores run this stencil at 3,803 steps/s (5.4 GFLOP/s, near-linear thread scaling), faster than the V3D's 2,127, so why involve the GPU at all? Because the deployment requirement is a real-time flood simulation running at the same time as the language query, and idle cores do not exist in that regime. The GPU's value is additive rather than comparative: its capacity does not come out of the inference budget. The moment decode runs, every CPU-only scheme pays steeply. Oversubscription is catastrophic: decode collapses 79% to 2.4 t/s, because llama.cpp's worker threads synchronize every token, and whenever any one of them loses its core to a flood thread, all four stall. Partitioning is the best CPU-only configuration and reaches (8.4 t/s, 681 steps/s); note that decode on 3 pinned cores alone slightly beats 4-core decode (11.8 against 11.4 in the same run: decode is memory-bound, and the fourth core adds contention rather than capacity), so its drop to 8.4 under partition is pure memory contention from one flood thread, a 29% tax against the GPU split's 10%. Time-slicing, derivable from the alone rates, loses on both axes too: matching the GPU's 712 steps/s average requires 18.7% of the time flooding, which caps average decode at 9.3 t/s and freezes guidance output entirely during each flood burst.

Against the best CPU-only alternative, the GPU-concurrent split delivers 23% more decode and 5% more simulation, and no CPU-only scheme beats it on either axis; it wins decisively

where the deployment is most sensitive (continuous language output) and never trails on simulation. The measured form of the inactive-silicon claim is: the GPU is slower than the CPU it sits next to, and the system is still strictly better for using it, because the CPU's cycles are already spoken for.

One note on thermals. The 1 Hz traces show that sustained decode engages the firmware's soft temperature limit on this passively cooled board even with no GPU work at all (in the reported run, 1 sample throttled during decode alone, 16 during each concurrent phase, peak 74.7 C; an earlier warmer-ambient run saw 14 to 31 throttled samples per sustained phase). Cool-throughout sustained measurements are not reliably attainable on this cooling, so the numbers above are steady-state, thermally governed figures rather than cold-silicon peaks. For the deployment question this paper asks, that is the right measurement: it is what Bonbibi actually gets. The flood-alone measurements, which complete before heat accumulates, are thermally clean, and the earlier isolated measurements (Section IV) agree with them. The CPU-counterfactual phases behave the same way: every sustained condition throttles at least briefly. Reproduction:

```
OUT=/tmp/conc_bench bash concurrency_bench.sh
OUT=/tmp/cpu_flood_bench bash cpu_flood_bench.sh
python3 analyze_conc_bench.py /tmp/conc_bench 4000
python3 analyze_conc_bench.py /tmp/cpu_flood_bench 4000
```

VII. What transfers and what does not

Three V3D findings from this and companion targets in the same harness, offered as portable heuristics for this class of GPU:

1. Optimize for the driver's register allocator first. v3dv compiles through a fallback ladder (disabling scheduling, then loop unrolling, then thread count) and the performance cliff is in the ladder, not in your arithmetic. Source-level de-unrolling rescued llama.cpp's matrix-vector kernel (+28% end-to-end decode); accumulator reduction took a GEMM kernel from 7.02 to 13.42 GFLOP/s in the same FSM under an NMSE gate.
2. Fixed per-invocation cost dominates small kernels. Strip-mining beat every memory-system optimization on the flood stencil, and 2 cells per invocation was the sweet spot before register pressure took the gain back.
3. Lane contiguity beats per-thread operation count. A transposed-B GEMM variant with fewer per-thread loads was verified correct and 36% slower.

The converse also holds: the de-unroll playbook did nothing for the flood stencil because its shaders never stressed the allocator. The harness's value is exactly that it prices each intuition on each target instead of letting the previous target's lesson ossify into doctrine.

VIII. Limitations

The performance envelope is specific: one board (Pi 5, V3D 7.1.10.2, Mesa v3dv 25.0.7), one grid family, and speedups that shrink as the grid grows (1.18x at 1024x1024). The optimized kernel still compiles through part of the v3dv fallback ladder, so headroom likely remains. The concurrency measurement uses llama-bench decode as the CPU load and the raster router as the latency probe; Bonbibi's street-graph CCH re-customization (about 14 ms per flood update on this board) was not separately measured under load. The CPU counterfactual was measured at 256x256 only, where the roughly 2 MB working set is cache-resident and thread scaling is near-linear; the CPU's raw-speed advantage may not survive grids that spill the cache. The LLM agent's proposals were not blind: the fused strip-2 kernel submitted in Section V's reproduction was discovered in Section IV's hand-driven sweep, so the MCP run demonstrates machine verification and verdict rather than de novo discovery. One scope note: the same harness's llama.cpp work on this GPU established that per-op correctness does not compose into end-to-end correctness at full offload due to an upstream driver-independent llama.cpp Vulkan defect we isolated on llvmpipe, so GPU LLM inference on this board currently holds only for a small-offload envelope; that is why Bonbibi keeps inference on the CPU and gives the GPU to physics.

IX. Relationship to the companion application

Bonbibi, the flood-guidance application, is a separately packaged project (github.com/msradam/bonbibi) that serves this paper as workload and demonstration vehicle. The contributions reported here, made beyond that application, are: the correctness-gated FSM harness and its MCP serving; the verified kernel-optimization ledger including the falsified hypotheses and the action-space result; the machine-checked reproduction; the concurrency envelope and its CPU-only counterfactual; and this paper's analysis. The application's interface, mobility-threshold research, and deployment materials are documented in the Bonbibi repository.

X. Reproducibility and availability

Everything is in two repositories: seppa (github.com/msradam/seppa: harness, FSM definitions, MCP server, driver scripts, kernels, and a running notes file with one entry per confirmed win and dead end) and Bonbibi (the application, github.com/msradam/bonbibi). The flood kit is `pi/flood/`: kernels, parameterized harness `vkflood2.cpp`, the falsification-sweep variants, and `concurrency_bench.sh`. The FSM target is `v3d_flood2_opt.py`, served by `theodosia_server.py --http --flood2`; `drive_flood2_mcp.py` reproduces Section V against that endpoint, and `replay_flood2.py` does the same in-process on the Pi. Raw logs for Section VI are produced by `concurrency_bench.sh` into a directory of plain-text files, from which every number in the table derives.

References

[1] M. Guidolin, A. S. Chen, B. Ghimire, E. C. Keedwell, S. Djordjevic, and D. A. Savic. A weighted cellular automata 2D inundation model for rapid flood analysis. Environmental Modelling & Software 84:378-394, 2016.

[2] J. Dibbelt, B. Strasser, and D. Wagner. Customizable Contraction Hierarchies. ACM Journal of Experimental Algorithmics 21, Article 1.5, 2016.

[3] A. Ouyang, S. Guo, et al. KernelBench: Can LLMs Write Efficient GPU Kernels? arXiv:2502.10517, 2025.
[4] B. Romera-Paredes et al. Mathematical discoveries from program search with large language models. Nature 625:468-475, 2024.
[5] Burr: build applications that make decisions. DAGWorks / Apache. https://github.com/DAGWorks-Inc/burr
[6] Model Context Protocol. https://modelcontextprotocol.io
[7] Bonbibi: offline, edge flood-aware accessible routing on a Raspberry Pi 5. https://github.com/msradam/bonbibi
[8] llama.cpp. ggml-org. https://github.com/ggml-org/llama.cpp
[9] AlphaEvolve: a coding agent for scientific and algorithmic discovery. Google DeepMind, 2025.
[10] V3D driver documentation, The Mesa 3D Graphics Library. https://docs.mesa3d.org/drivers/v3d.html
[11] C. Baronio, P. Marsella, B. Pan, S. Alberti, S. Ehrlich et al. Kevin: Multi-Turn RL for Generating CUDA Kernels. arXiv:2507.11948, 2025.
[12] X. Li et al. CUDA-L1: Improving CUDA Optimization via Contrastive Reinforcement Learning. arXiv:2507.14111; ICLR 2026.
[13] TritonRL: Training LLMs to Think and Code Triton Without Cheating. arXiv:2510.17891, 2025.
[14] B. Zhao, D. Srikanth, Y. Wu, and Z. Jiang. SpecBench: Measuring Reward Hacking in Long-Horizon Coding Agents. arXiv:2605.21384, 2026.
[15] D. Sarkar. The Correctness Illusion in LLM-Generated GPU Kernels. arXiv:2606.20128, 2026.
[16] J. Blocklove, S. Garg, R. Karri, and H. Pearce. Chip-Chat: Challenges and Opportunities in Conversational Hardware Design. ACM/IEEE Workshop on Machine Learning for CAD (MLCAD), 2023.
[17] S. Thakur, B. Ahmad, H. Pearce, B. Tan, B. Dolan-Gavitt, R. Karri, and S. Garg. VeriGen: A Large Language Model for Verilog Code Generation. ACM TODAES; arXiv:2308.00708, 2023.